Protocol: Adapting the Repository Toward the Temeraire Redis Experiment

May 4, 2026

1 Purpose

This note documents what had to be adjusted so that the repository more closely reflects the Redis case-study experiment from the OSDI'21 Temeraire paper (*Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator*). It is not a full reproduction of the paper, because the paper also contains Google-internal production and fleet-scale evaluation that cannot be reproduced in this environment. The goal of this protocol is to record:

- what the repository originally did,
- why that was not yet a close paper reproduction,
- what we changed,
- what failed during setup,
- and what final state now works.

2 Initial Repository State

At the beginning, the repository was a useful scaffold for allocator experiments, but it was not yet aligned with the paper's Redis experiment. In particular:

- the setup used Redis `7.2.5` instead of Redis `6.0.9`,
- the comparison was primarily `glibc` versus open-source `gperftools tcmalloc`,
- there was no Temeraire versus legacy pageheap comparison,
- the README could be read as if this might reproduce the paper more directly than it actually did.

After checking the paper text, it became clear that the public seminar reproduction should focus on the Redis case study only, and should be explicit about that scope.

3 Paper-Relevant Clarifications

From the paper, the most relevant constraints for this repository are:

- the Redis experiment uses Redis `6.0.9`,
- the important allocator comparison is `Temeraire` versus the legacy `TCMalloc` pageheap,

- the workload consists of repeated list operations: pushing five elements and then reading those five elements,
- the paper reports `2000` trials with `1,000,000` requests per trial,
- the fleet experiment and production rollout are not reproducible here.

This led to the main design decision: the repository should be reframed as a small-scale reproduction of the Redis case study methodology, not as a reproduction of the full paper evaluation.

4 README Changes

The first adjustment was editorial. The README was updated so that it now:

- states explicitly that this repository is not a reproduction of the fleet-scale part of the paper,
- explains that the relevant local target is the Redis case study,
- clarifies that the desired allocator comparison is legacy pageheap versus Temeraire,
- documents that historical public `google/tcmalloc` code is used to approximate the paper-era setup.

This clarification is important for later seminar work because it prevents overclaiming what the artifact reproduces.

5 Build and Environment Adjustments

5.1 Docker and Toolchain

The container environment was adjusted to support the paper-shaped workflow:

- the image base was kept explicit as `debian:bookworm-slim`,
- Redis was pinned to `6.0.9`,
- `gperftools` was pinned to `2.16`,
- `clang` was added to the container toolchain,
- defaults for trial count, requests per trial, client count, and pipeline depth were added to `docker-compose.yml`.

The important point is that the container is no longer just a generic Linux environment; it now encodes a closer approximation of the paper's Redis experiment. The Linux environment still has two layers that must be documented separately: Debian bookworm user space from the image, and the shared Linux kernel supplied by Docker/WSL2 or the host. The latter controls THP behavior, kernel version, cgroup behavior, and hardware-counter availability, so it is part of the experimental metadata rather than a property fixed by the Dockerfile.

5.2 Historical tcmalloc Source

The major technical step was adding a historical public `google/tcmalloc` checkout. The paper points to the TCMalloc repository, but the current repository has evolved beyond the paper artifact. To get closer to the paper era, a historical commit was needed.

The chosen commit was:

```
8e534f50707469baac732559494559db95732e12
```

This commit was chosen because it still contains the mechanism needed to disable HPAA/Temeraire-style hugepage-aware behavior via `want_no_hpaa`, which makes a legacy versus Temeraire split possible in the public code.

6 Setup Script Changes

The setup script was extended substantially. It now:

- re-syncs repositories to pinned references even if they already exist locally,
- clones Redis `6.0.9`,
- clones `gperftools 2.16`,
- clones historical `google/tcmalloc`,
- installs `bazelisk` when missing,
- builds Redis with `clang`,
- builds two allocator shared libraries from the historical TCMalloc source:
  - a Temeraire-capable shared object,
  - a legacy shared object with `want_no_hpaa` linked in.

These libraries are copied into a stable install directory so that benchmark scripts can switch allocators through `LD_PRELOAD`.

6.1 Exact LLVM Integration

The paper states that Redis and TCMalloc were compiled with LLVM commit `cd442157cf` using `-O3`. The repository initially did not try to match this: it simply relied on the container's packaged `clang`. To move closer to the paper, the setup script was extended with an optional exact LLVM path.

When enabled, the script now:

- clones an LLVM repository at a configurable reference,
- builds a local `clang`/`clang++` toolchain into `third_party/install/llvm`,
- uses that compiler explicitly for Redis, `gperftools`, and the TCMalloc wrapper build,
- records the resulting compiler version and pinned LLVM reference in the collected system metadata.

This is an important methodological improvement, and it is no longer only theoretical. In the later paper-closer rerun, the collected system metadata shows that Redis and the allocator wrapper were in fact built with a locally built LLVM 13.0.0 toolchain from commit `cd442157cff4aad209ae532cbf031abbe10bc1df`. In other words, the repository now supports not only the paper-shaped workload but also a paper-closer compiler path that has been exercised successfully end to end.

One inconsistency appeared in the Docker/WSL manifests. They set the `exact_llvm_commit_cd442157cf` flag to `no`, even though the associated system-information files confirm that the exact LLVM path was used in those runs. For those runs, trust the collected compiler metadata over the manifest flag. The later bare-metal manifests set the same flag to `yes`, so flag and metadata agree from the node85 stage onward.

7 Benchmark Script Changes

The benchmark driver had to be reoriented toward the paper's Redis case study. The new script behavior is:

- allocator modes now include `legacy` and `temeraire`,
- Redis is launched with the selected allocator through `LD_PRELOAD`,
- each trial performs the paper-shaped list operations:
  - `LPUSH` of five values,
  - `LRANGE 0 4`.
- per-trial and summary CSV output is generated,
- the old `glibc`/`gperftools` comparison path remains available for side experiments, but is no longer the main paper-aligned comparison.

The paper's exact surrounding environment cannot be reproduced, but this script now matches the experiment shape more closely than the original repository state.

7.1 Single-Machine Paper-Closer Orchestration

Beyond the original benchmark script, the repository now also includes a dedicated orchestration script for a "paper-closer" single-machine run. This script is intended to make the experiment easier to execute consistently when trying to match the Redis case study more closely.

The orchestration path adds several controls that were previously missing:

- optional NUMA pinning through `numactl`,
- optional `perf` collection for the same allocator mode,
- a run manifest that explicitly records known deviations from the paper,
- separate release-mode runs so that "release off" and "release on" configurations can be documented distinctly.

This is valuable for a seminar artifact because it makes the difference between the *paper-shaped workload* and the *paper-shaped workflow* explicit.

8 Dead Ends and Failed Attempts

The adaptation process involved some trial and error. This is worth documenting because these failures explain why the final solution looks the way it does.

8.1 Modern TCMalloc Build Attempt

The first idea was to build a modern `google/tcmalloc` checkout and wrap it externally via CMake. That path failed because upstream CMake/test-linkage behavior around whole-archive handling caused build conflicts. In other words, the modern repository was buildable in general, but not in a convenient way for this exact wrapper strategy.

8.2 External Bazel Wrapper Workspace

The second idea was to build a wrapper in a separate Bazel workspace. That also failed, because the wrapper did not naturally inherit all of TCMalloc's workspace dependencies, such as external Abseil references.

8.3 What Bazel Helped With

Bazel was useful precisely because historical TCMalloc is already organized as a Bazel project. Once the wrapper target was placed inside the cloned TCMalloc tree, Bazel supplied the allocator's native dependency graph, build settings, and link semantics without requiring a separate reconstruction in CMake or Make. It also made the allocator comparison narrow: both output libraries are produced from the same wrapper source, while the legacy variant adds `//tcmalloc:want_no_hpaa` and the Temeraire-capable variant does not.

The same choice created several practical problems:

- Bazelisk had to be installed in the container and pinned through `USE_BAZEL_VERSION` so the historical checkout would build consistently.
- The exact LLVM compiler path had to be passed through Bazel using `CC`, `CXX`, `-repo_env`, and `-action_env`; otherwise Redis and TCMalloc could silently be built by different compilers.
- An external Bazel workspace was not sufficient, because the small wrapper did not automatically inherit all of TCMalloc's repository-local and external dependencies.
- Target visibility mattered: `//tcmalloc:want_no_hpaa` was not linkable from an arbitrary package, so the wrapper target had to live in `tcmalloc/testing`.

In short, Bazel made the final build more faithful to the allocator's native structure, but only after the wrapper was moved into the right package and the compiler environment was made explicit.

8.4 Visibility Constraint

Another obstacle was target visibility. The `want_no_hpaa` mechanism was not globally visible; it was restricted in a way that prevented arbitrary external linkage. The workaround was to place the wrapper target inside the `tcmalloc/testing` package, where the needed visibility was permitted.

8.5 Legacy Comparison Was Not the Default Shape

An earlier practical difficulty was conceptual rather than purely technical: the repository was capable of allocator experiments, but its main comparison axis was not the one used in the paper. This matters because systems artifacts often become harder to interpret when a repository supports many adjacent experiments at once. Part of the work was therefore not just adding code, but removing ambiguity about which allocator comparison is actually paper-aligned.

8.6 Exact Compiler Matching Is More Expensive Than It First Appears

It is easy to underestimate the cost of matching a paper's compiler statement. In this case, "use LLVM commit `cd442157cf`" sounds small, but in practice it implies:

- identifying the correct upstream repository layout for that historical commit,
- installing additional build dependencies into the container,
- compiling a local `clang` toolchain before the real experiment can even start,
- passing that compiler through three different build systems: Redis `make`, `gperftools` `configure`/`make`, and Bazel for the TCMalloc wrapper.

This does not make the approach infeasible, but it does make it one of the largest setup-cost increases in the repository.

8.7 Wrapper Symbols Did Not Match the Historical TCMalloc Revision

During a later container validation run, the allocator preload check failed even though the shared objects had been built successfully. The immediate cause was that the small wrapper library referenced background-release methods on `tcmalloc::MallocExtension` that do not exist in the pinned historical public TCMalloc revision. As a result, the preloaded allocator library failed to load before Redis even started.

The practical fix was to make those wrapper calls optional by resolving the symbols dynamically at runtime. This preserved the wrapper's intended behavior for newer revisions that provide the methods, while allowing the historical paper-era approximation to load cleanly when those methods are absent.

8.8 Why the Final Approach Worked

The successful strategy was therefore:

- use a historical public commit that still exposes legacy-versus-Temeraire behavior,
- add a small wrapper source file directly into the cloned TCMalloc tree,
- add Bazel targets in a package where the required visibility is valid,
- produce two small shared libraries that can be switched with `LD_PRELOAD`,
- and push paper-specific knobs such as compiler choice, release mode, and NUMA placement into explicit script parameters instead of leaving them as undocumented ambient state.

9 What Worked Surprisingly Well

Not every part of the adaptation was difficult. Several parts worked better than expected and are worth documenting because they strengthen confidence in the artifact design.

9.1 Allocator Switching by `LD_PRELOAD`

Once the two shared libraries were built, allocator switching itself turned out to be straightforward. The benchmark harness could keep a single Redis build and select the allocator mode externally with `LD_PRELOAD`. This greatly reduced the amount of per-mode rebuild logic required.

9.2 The Historical TCMalloc Commit Was Sufficiently Usable

The chosen historical public TCMalloc revision did not merely contain `want_no_hpaa`; it was also practical enough to support the legacy-versus-Temeraire split without a deep fork of allocator internals. That was an important positive surprise because it kept the repository from turning into a TCMalloc maintenance branch.

9.3 Metadata Collection Fit the Artifact Model Well

Collecting system metadata, allocator preload evidence, and raw benchmark outputs fit naturally into the repository structure. In other words, once the repository was already writing timestamped result directories, it was easy to add more context such as the Debian user-space version, shared kernel release, container markers, THP settings, NUMA information, and compiler version records. This is useful because reproducibility problems in systems experiments often come less from one wrong command than from one unrecorded assumption.

10 Verification Steps

Several checks were used to verify that the adapted repository actually works.

10.1 Container Build

The following commands succeeded:

```
docker compose build
docker compose run --rm temeraire-dev bash -lc "./scripts/setup_env.sh"
```

This confirmed that the complete toolchain, source checkouts, and allocator builds can be reproduced in the container.

10.2 Allocator Artifacts

After setup, the following shared libraries existed:

```
third_party/install/google-tcmalloc/lib/libtcmalloc_legacy.so
third_party/install/google-tcmalloc/lib/libtcmalloc_temeraire.so
```

10.3 Smoke Benchmark

A small smoke test was run with reduced trial counts and request counts before attempting any large run. This verified that both allocator modes could execute end-to-end through the benchmark harness.

10.4 Runtime Preload Verification

An additional check script launched Redis under both allocator modes and inspected `/proc/$pid/maps`. This showed that:

- `legacy` mode actually loaded `libtcmalloc_legacy.so`,

- `temeraire` mode actually loaded `libtcmalloc_temeraire.so`.

This matters because Redis may still report `mem_allocator: libc` from its own build metadata, even though the process allocator has been changed at runtime via `LD_PRELOAD`. The preload verification therefore provides stronger evidence that the allocator switch is real.

10.5 Script Validation

After the later harness extensions, the modified shell scripts were at least validated syntactically with `bash -n`. This is weaker than a full container rebuild, but it is still worth documenting because the repository now contains more branching behavior than before: exact LLVM builds, NUMA pinning, release-mode toggles, and paper-closer orchestration.

10.6 Paper-Closer End-to-End Runs

That later orchestration path was also exercised end to end. The repository now contains complete raw result directories for:

- a plain 2000-trial `legacy` run and a matching `temeraire` run,
- a paper-closer `release off` pair,
- and a paper-closer `release on` pair.

This means the key artifact paths are now verified by actual benchmark output rather than only by static script inspection.

11 Current Final State

In its current state, the repository now supports a more faithful small-scale approximation of the paper's Redis case study:

- Redis version aligned to `6.0.9`,
- historical public TCMalloc source included,
- legacy and Temeraire allocator variants built as shared objects,
- benchmark script built to one reading of the push-five/read-five workload shape,
- Docker defaults prepared for repeated trials,
- runtime verification added to ensure the chosen allocator is truly loaded,
- richer metadata collection for CPU, NUMA, THP, and compiler details,
- per-run THP-state capture before and after Redis benchmark runs, plus periodic `smaps_rollup` snapshots during long runs,
- exact-LLVM integration toward the paper's stated compiler setup, now exercised in a completed paper-closer rerun,
- and a paper-closer orchestration path for a single-machine experiment.

This is likely sufficient for a seminar artifact that aims to be methodologically close to the Redis experiment, provided the report clearly states what remains outside the scope of reproduction.

12 Noteworthy Results

The completed runs are worth documenting here because they affect how the artifact should be interpreted.

12.1 Result Pattern Across Run Families

The result pattern is best understood in two phases.

First, earlier paper-closer runs were completed while the shared WSL2 kernel still exposed Transparent Huge Pages in `madvise` mode. Those runs were useful for debugging the workflow, but their process summaries showed `AnonHugePages: 0 kB`, so they did not provide strong evidence that the paper's hugepage mechanism had actually been exercised.

Second, a later four-run paper-closer rerun was performed after explicitly switching the shared WSL2/Docker THP policy to `always`. That rerun is the important one for the final interpretation.

The run families are therefore:

- an earlier plain legacy-versus-Temeraire comparison,
- an earlier paper-closer pair in `madvise`-mode conditions,
- and a final THP-enabled paper-closer pair with periodic release disabled plus a matching pair with periodic release enabled.

Using a combined push-plus-read throughput metric, the observed Temeraire change versus legacy in the final THP-enabled rerun was:

- about `+1.88%` in the paper-closer release-off run,
- and about `+0.26%` in the paper-closer release-on run.

For context, the earlier local runs had produced much less credible values from the perspective of the paper comparison: about `+5.76%` for release-off and about `-0.01%` for release-on. Once THP backing was actually present, the observed Temeraire effect became smaller and moved closer to the paper's own reported range.

12.2 Comparison With the Paper

Hunter et al. report two Redis rows in Table 1: `Redis`† (*periodic release off*) at `+0.75%` and `Redis` (*periodic release on*) at `+0.44%`. The local release-on result at `+0.26%` is now close to the paper's small positive effect. The local release-off result at `+1.88%` is still somewhat larger than the paper's `+0.75%`, but it is much closer than the earlier local `madvise`-mode result was.

12.3 Hugepage Mechanism Was Recovered in the Final Rerun

The most important methodological improvement is that the final rerun did exercise hugepage-backed anonymous memory. The new periodic `memory-sample-XXXX.txt` snapshots and the final `memory-after.txt` files show non-zero `AnonHugePages` during and after every completed run.

In the final THP-enabled rerun, the end-state `AnonHugePages` values were roughly:

- `18 MiB` for legacy release-off,
- `14 MiB` for Temeraire release-off,
- `14 MiB` for legacy release-on,

- `20 MiB` for Temeraire release-on.

This does not mean the paper's exact hugepage-coverage numbers were reproduced. The host still ran under WSL2 on an Intel Core i7-10700K rather than on the paper's Intel Skylake Xeon servers, so the kernel and platform behavior remain different. But it does mean that the final local comparison is now about the same qualitative mechanism as the paper, rather than about a benchmark that never obtained hugepage backing at all.

12.4 Instrumentation Added After the Hugepage Hiccup

The first completed runs revealed a methodological blind spot. The harness recorded process memory state only before and after the full Redis benchmark. When those snapshots showed `AnonHugePages: 0 kB`, there were two possible interpretations:

- hugepages were never used at all,
- or hugepages were used transiently during the run but were absent by the time the final snapshot was collected.

To reduce that ambiguity in future reruns, the benchmark harness was extended after these results were collected. It now writes:

- `thp-before.txt` and `thp-after.txt` for each benchmark run,
- a richer end-state `memory-after.txt`,
- and periodic `memory-sample-XXXX.txt` snapshots during the benchmark, with the interval configurable through `REDIS_SNAPSHOT_EVERY_TRIALS` or `PAPER_SNAPSHOT_EVERY_TRIALS`.

This change did not retroactively fix the already collected runs, but it did make the next rerun much better suited to answer the narrower question: whether Temeraire ever obtained hugepage-backed anonymous memory on this host during the benchmark itself. The answer from the later THP-enabled rerun is now yes.

12.5 Balanced Reruns and Current Release-On Anomaly

After the initial THP-enabled paper-closer comparison, additional balanced paper-closer reruns were started to test whether the small effect survives allocator order changes. The balanced driver alternates which allocator is measured first within each release-mode pair.

The first two balanced reruns still pointed in the paper's direction: approximately `+0.43%` and `+0.48%` for release-off, and approximately `+0.12%` and `+0.38%` for release-on. A third balanced rerun then produced negative deltas, approximately `-0.76%` for release-off and `-2.34%` for release-on. A fourth balanced rerun was especially mixed: release-off strongly favored Temeraire at about `+3.46%`, while release-on strongly disfavored Temeraire at about `-12.02%`.

The fourth rerun's release-on Temeraire result is treated as an anomaly rather than folded silently into the final interpretation. Its Redis log contains no application error. The run was correctly marked `paper-release-on`, background release was enabled, THP was still `always`, and `smaps_rollup` samples showed stable RSS and stable `AnonHugePages` throughout.

The slowdown was visible from the first 250-trial window and only partially recovered near the end of the run, while the paired legacy release-on run was stable. This pattern suggested transient host, VM, or scheduling interference more than progressive Redis memory growth or THP collapse.

A targeted release-on-only rerun with Temeraire first was then performed:

```
docker compose run --rm temeraire-dev bash -lc "echo always > /sys/kernel/mm/
    transparent_hugepage/enabled && echo always > /sys/kernel/mm/transparent_hugepage/
    defrag"
docker compose run --rm -e RUN_RELEASE_OFF=0 -e RUN_RELEASE_ON=1 temeraire-dev bash -
    lc "./scripts/run_paper_closer_redis_experiment.sh --allocator-order temeraire-
    first"
```

That targeted rerun returned to the earlier near-flat `release-on` range. Its combined push-plus-read delta was approximately `+0.12%`: `-0.04%` for LPUSH and `+0.37%` for LRANGE. The Temeraire run was also stable across 250-trial windows, with combined throughput around `940-954 kRPS`. This makes the fourth balanced rerun's `-12.02%` release-on result best treated as a transient outlier rather than as representative evidence that `release-on` Temeraire is consistently bad on this host.

The current working summary for later report editing is:

Run	Release-off	Release-on	Note
THP fixed-order	`+1.88%`	`+0.26%`	first valid THP run
balanced 1	`+0.43%`	`+0.12%`	legacy first
balanced 2	`+0.48%`	`+0.38%`	Temeraire first
balanced 3	`-0.76%`	`-2.34%`	negative run
balanced 4	`+3.46%`	`-12.02%`	release-on outlier
targeted release-on	n/a	`+0.12%`	rerun of anomaly

The interpretation available at this point was cautious: release-off had a recurring positive signal in the paper's direction, while release-on was small and noisy.

Superseded. Two later findings retract the reading above. The audit in Section 16 shows that every `release-on` value in this table ran at a zero background release rate, so that whole column measures something other than periodic release. The bare-metal run in Section 17 then reverses the release-off conclusion: its median is -0.34%, not positive. Read this table as a record of the development stage only.

13 Remaining Limitations

The following limitations still apply and should be stated explicitly in seminar material:

- the experiment is not run on Google production hardware,
- the fleet-scale and rollout sections of the paper are not reproduced,
- the exact compiler, kernel, and machine topology from the paper are only approximated,
- containerization may itself affect allocator and hugepage behavior,
- the public TCMalloc source is only a historical approximation of the paper artifact, not a guaranteed byte-identical reproduction of the internal setup,
- exact-LLVM matching only reduces one class of deviation; it does not remove the hardware and operating-system mismatch,
- and although the final rerun did show hugepage-backed anonymous memory, the exact hugepage coverage, physical fragmentation state, and host-kernel behavior still differ from the paper's environment.

These limitations do not invalidate the artifact, but they define its proper claim: it is a careful public reproduction attempt of the Redis case-study portion of the paper.

14 Recommended Seminar Framing

A suitable and honest framing for the seminar is:

> This artifact reproduces the methodology of the Redis case study from the Temeraire paper as closely as possible with public code and commodity hardware, while explicitly not claiming reproduction of the paper's fleet-scale production evaluation.

15 Minimal Working Commands

The following command sequence captures the final working workflow:

```
docker compose build
docker compose run --rm temeraire-dev bash -lc "./scripts/setup_env.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/check_allocator_preload.sh"
docker compose run --rm temeraire-dev bash -lc "echo always > /sys/kernel/mm/
    transparent_hugepage/enabled && echo always > /sys/kernel/mm/transparent_hugepage/
    defrag"
docker compose run --rm temeraire-dev bash -lc "./scripts/
    run_paper_closer_redis_experiment.sh"
```

The older direct benchmark path remains available for supplementary runs:

```
docker compose run --rm temeraire-dev bash -lc "./scripts/collect_system_info.sh"
docker compose run --rm temeraire-dev bash -lc "./scripts/run_redis_benchmark.sh
    legacy"
docker compose run --rm temeraire-dev bash -lc "./scripts/run_redis_benchmark.sh
    temeraire"
```

For smoke testing, the environment variables for trial count and request count can be lowered temporarily. For seminar measurements, the defaults should remain documented and any deviations should be recorded.

16 Post-Audit Release-On Finding

A final audit identified a configuration error in every run labelled `release-on`. The orchestration enabled TCMalloc's background-actions thread, but it did not set the background-release-rate environment variable (`TEMERAIRE_TCMALLOC_BACKGROUND_RELEASE_RATE_BPS`). In the pinned public TCMalloc revision, the default background release rate is zero. The thread therefore continued to perform its per-CPU-cache maintenance, but its pageheap release calculation produced zero bytes and did not invoke an effective periodic `ReleaseMemoryToSystem` operation.

Consequently, the existing `release-on` measurements do not reproduce the paper's periodic-memory-release condition and must not be compared with the paper's `+0.44%` Redis result. This applies both to the balanced runs and to the targeted rerun of the `-12.02%` anomaly. Those logs should be retained as evidence of the discovered configuration, but their earlier interpretation as a valid release-on experiment is superseded by this finding.

Before rerunning this condition, the scripts should require and record an explicit positive background release rate and should fail when `release-on` is requested without one. Because the paper does not state the release rate used for Redis, any corrected run must report its chosen rate as an additional experimental assumption; ideally, several documented rates should be tested as a sensitivity analysis. The release-off runs are not invalidated by this issue, although their small sample and run-to-run variance still require cautious, descriptive interpretation.

16.1 What the correction exposed

Setting a non-zero rate was not enough to make the corrected run work. The first smoke test aborted at startup because the wrapper tried to find `SetBackgroundReleaseRate` and `ProcessBackgroundActions` with `dlsym`, but those symbols were not available through `RTLD_DEFAULT` in the Redis process. The old wrapper had hidden this problem by returning silently when a lookup failed. The corrected wrapper now calls the TCMalloc API directly, and both Bazel targets explicitly link `//tcmalloc:malloc_extension`. It also writes the applied byte rate to the Redis server log. The benchmark refuses to continue unless that exact log line is present.

The same smoke test review found that `LD_PRELOAD` had been exported by the benchmark script. It therefore applied not only to `redis-server`, but also to `redis-cli` and `redis-benchmark`. This mixed server and client allocator effects into the old comparison. Preloading is now scoped to the server command, so the benchmark client is identical in the legacy and Temeraire conditions.

Rebuilding then exposed an older naming mismatch in the checked-out TCMalloc tree: its existing targets were named `codex_*`, while the current setup script creates `temeraire_*` targets. The installed libraries had to be rebuilt from the targets that actually existed, while the setup template was updated with the missing link dependency. Finally, several shell scripts still had Windows CRLF line endings, causing Linux to look for `bash\r`. Normalizing them to LF was necessary before the long run could start.

These failures changed how the corrected experiment is validated. Release-on now requires a positive rate before any metadata or benchmark work begins. A two-trial smoke run must succeed for both allocators, the Redis log must confirm the requested rate, and `LD_PRELOAD` must remain confined to the server. Only after those checks did the full 16, 64, and 256 MiB/s sensitivity matrix start.

16.2 Completed corrected release-on matrix

The corrected matrix completed all 24 allocator runs: four legacy/Temeraire pairs at each of 16, 64, and 256 MiB/s, with 2000 trials per allocator run. Allocator order alternated within each rate. Every Redis server log confirmed the requested positive byte rate before the benchmark proceeded.

Rate	Pair 1	Pair 2	Pair 3	Pair 4	Median
16 MiB/s	+1.91%	−0.16%	−0.27%	−1.37%	−0.21%
64 MiB/s	−6.88%	+2.10%	+11.49%	+12.21%	+6.80%
256 MiB/s	+15.73%	−0.09%	+0.50%	−0.77%	+0.20%

The rates do not form a monotonic pattern. The 16 MiB/s pairs are centered near zero. Three of the four 256 MiB/s pairs are also within one percentage point of zero; its +15.73% first pair is an outlier. The 64 MiB/s runs range from −6.88% to +12.21%. During that part of the matrix, absolute combined throughput moved from roughly 978 kRPS down to 753 kRPS and later recovered above 950 kRPS. Similar recovery explains the first 256 MiB/s pair. The allocator that happened to run during a slow or recovered host period inherited most of that difference.

The corrected experiment therefore does not show a stable release-rate effect or robustly reproduce the paper's +0.44% release-on result. Its useful result is methodological: release is now verifiably active, while the remaining single-host Docker/WSL2 drift is larger than the sub-one-percent effect being measured. Future work should interleave shorter allocator blocks or run both conditions on isolated, controlled hosts instead of comparing consecutive hour-long runs.

17 Bare-Metal Rerun on node85

The Docker/WSL2 drift was the reason for a second workflow rather than another container run. The native scripts keep the same pinned source revisions, allocator wrappers, Redis workload, result layout, and pair structure, and run directly on a dedicated Debian 13 node.

17.1 Recorded environment

Field	Recorded value
OS	Debian GNU/Linux 13 (trixie)
Kernel	`6.12.95+deb13-amd64`
Processor	Intel Xeon Gold 5318N
Topology	24 cores, 48 threads, one NUMA node
Memory	188 GiB
Virtualization	none detected
THP	`always`; defrag `madvise`

17.2 New practical problems

The move to the cluster produced a fresh set of obstacles, none of them related to the allocator:

- The shared home directory was too slow for Bazel's many small files, so the build and the runs were moved to `/var/tmp`.
- The compute node could not fetch the pinned sources, so the source and dependency archives had to be staged for an offline build. Each staged directory carries a `.temeraire-source-ref` marker that `setup_bare_metal_env.sh` checks.
- The paper-era LLVM revision needed `-include cstdint` to build with Debian 13's host compiler.
- The German numeric locale wrote decimal commas into `summary.csv` and broke its three-column layout. The native runner now sets `LC_ALL=C`. Two early two-trial smoke blocks predate that fix and still contain the broken files; the audit script excludes them because they do not have 2000 trials.

17.3 Runs completed

Three two-trial smoke runs and two ten-trial pilots came first. Four full balanced pairs then ran at 16 MiB/s, each pair covering legacy and Temeraire with release off and release on. A second series kept release enabled and added four order-alternated pairs at 64 MiB/s and four at 256 MiB/s. Every full allocator block holds 2000 trials, and every release-on Redis log confirms the requested rate before the benchmark proceeds.

17.4 Results

Condition	Mean	Median	95% interval
Release off	-0.28%	-0.34%	$[-1.04\%, +0.48\%]$
Release on, 16 MiB/s	$+0.51\%$	$+0.55\%$	$[-0.69\%, +1.73\%]$
Release on, 64 MiB/s	-0.49%	-0.60%	$[-0.92\%, -0.07\%]$
Release on, 256 MiB/s	-0.29%	-0.08%	$[-1.54\%, +0.96\%]$
Paper, release off	$+0.75\%$		
Paper, release on	$+0.44\%$		

Each interval is a Student-t interval on the log-transformed pair ratios, with a complete run pair as the replication unit. It therefore belongs to the mean rather than to the median beside it, and it does not treat the 2000 sequential trials inside a block as independent replications.

Three points follow. First, the node removes the double-digit swings: all sixteen blocks of the main run landed between 989 and 1003 kRPS combined, a total spread of about 1.4%, against the 200 kRPS movement seen under WSL2. Second, the release-off conclusion from the Docker/WSL2 stage does not survive. Its native interval excludes the paper's +0.75%, so the aggregate is inconsistent with the paper's release-off row rather than merely quieter than it. Third, the release-on agreement at 16 MiB/s holds at one rate only. The 64 MiB/s interval is the only one here that excludes zero, and it does so on the legacy side; with four pairs, a normal approximation, and four conditions tested without correction, that is worth recording rather than treating as an established effect.

Two comparability limits apply to the whole table. The local metric is raw `redis-benchmark` throughput with no normalization for CPU time, joined across two operations by a harmonic mean that the paper does not specify. And the paper's one-million-request trial "to push 5 elements and read those 5 elements" admits more than one implementation; the sequential form used here issues one million pushes and then one million reads, two million commands in total, with every push preceding every read. A percentage that matches the paper is therefore proximity between differently built quantities.

17.5 Audit

The audit script re-derives both halves of the archive from the raw trial files. It lives at `scripts/` and uses only the Python standard library. Run it in two modes:

```
python3 scripts/audit_bare_metal_results.py --mode balanced \
  --raw-dir results/node85-import/raw \
  --archive temeraire-node85-results.tar.gz \
  --checksum temeraire-node85-results.tar.gz.sha256 \
  --output-dir results/processed/node85-audit

python3 scripts/audit_bare_metal_results.py --mode sensitivity \
  --raw-dir results/node85-sensitivity-audit/raw \
  --archive temeraire-node85-results-with-sensitivity.tar.gz \
  --checksum temeraire-node85-results-with-sensitivity.tar.gz.sha256 \
  --output-dir results/processed/node85-sensitivity-audit
```

The script verifies the archive checksum, then checks trial identifiers, request counts, raw file counts, memory samples, recomputed block means, THP state, release-rate confirmations, clean shutdowns, allocator order, and system metadata. Any mismatch stops it with a non-zero exit code. Both commands pass on the current archives.

17.6 Limitations that remain

The bare-metal stage removes the container, the WSL2 kernel, and the competing desktop load. It does not remove the rest:

- Four pairs per condition is a small sample for sub-one-percent effects.
- Allocator blocks run consecutively, so slow changes in frequency, temperature, or system activity can survive inside a pair.
- The metadata omits CPU governor, turbo state, temperature, and per-block allocator mappings. Preload was verified once before the run instead of inside every result directory.

- The paper does not state a background release rate for Redis, so every release-on rate here is a local experimental parameter.
- The Xeon Gold 5318N and the Debian 13 stack still differ from the paper's environment, and the public TCMalloc revision remains a historical approximation of Google's internal allocator.